Pauli Frame Sampling on Four FPGA Nodes: A Pipelined Stabilizer Circuit Sampler for Quantum Error Correction on Alveo U55C Accelerators

Nasir Ali[1*]

[1*]Embedded Systems Division, Centre for Development of Advanced Computing, Noida, 201307, India.

Corresponding author(s). E-mail(s): nasirali2607@gmail.com;

Abstract

Estimating thresholds and benchmarking decoders for quantum error correction requires sampling very large numbers of detector outcomes from noisy stabilizer circuits, today mostly with the Stim library on CPUs. We present a Pauli frame sampler for the AMD Alveo U55C that processes one instruction slot per clock cycle for 64 shots at a time, and we run it on four cards in one host that are treated as independent nodes, as in our earlier multi FPGA statevector simulator. Three changes remove the bottlenecks of a first single card kernel: noise masks are produced by a separate pipelined stage from a multiplier free counter based generator, the Pauli frame of up to 128 qubits is updated through a one read, one write pipeline whose hazards the compiler removes by scheduling, and detectors are folded from a per batch measurement record while the next batch runs. Each card holds 2 kernel instances in separate super logic regions, and every node, instance and batch draws its own random stream, so the nodes exchange no data. The kernel is bit exact against a software model in C simulation, in RTL cosimulation and on every card, and its detector rates and decoded logical error rates agree with Stim for surface codes up to distance 7. On four cards the sampler delivers 61.4 million shots per second for a distance 5 surface code, against 724 thousand for one Stim process and 57.2 million for 256 Stim processes on the host's 256 hardware threads.

Keywords: FPGA, Quantum error correction, Stabilizer circuit simulation, Pauli frame, Multi-FPGA, High-level synthesis, Alveo U55C, Surface code

1 Introduction

Quantum error correction protects logical information by measuring stabilizers of a code repeatedly and decoding the resulting detector events. The surface code [1–3] is the leading candidate, and recent experiments have shown logical error rates that fall as the code distance grows [4, 5]. Designing such experiments, estimating thresholds and benchmarking decoders [6–8] all depend on Monte Carlo sampling: a noisy stabilizer circuit is sampled millions to billions of times, and every sample (a shot) yields one bit per detector and per logical observable. The statistical error of a logical error rate p_L estimated from N shots is about $\sqrt{p_L/N}$, so resolving the small error rates that matter for fault tolerance needs very many shots.

Stim [9] made this workload fast on CPUs. It computes one noiseless reference sample of the circuit with a stabilizer tableau [10, 11] and then samples noisy shots by propagating only the Pauli frame [12], the deviation of each shot from the reference, through the circuit. A frame update is a few bitwise operations per gate, applied to many shots packed into one machine word. The work is therefore regular, branch free and independent between shots: an obvious match for a deeply pipelined hardware datapath with wide words, and also for many devices working side by side.

This paper treats that observation as a systems question for FPGA accelerators. In earlier work we built a distributed statevector simulator on four Alveo U55C cards in one host, in which every card is a node that holds a slice of the state, and amplitudes move between nodes through host memory [13]. Treating the cards as the nodes of a small cluster, with the host acting as interconnect and scheduler, is the model that let four cards work as one machine there, and it is the model we adopt here. Pauli frame sampling fits the same node model, with one important difference: shots are independent, so nodes never need to exchange data. The questions are then how fast one node can run, whether four nodes scale without interfering through the shared host, and how the result compares with Stim running on all cores of the same host.

A first single card kernel for this workload, which we built earlier in this project and which has not been published (Section 4.2), validated the approach end to end on hardware but was slower than one Stim process on every circuit tested, because its instruction loop was not pipelined and each launch sampled only 64 shots. Here we redesign the kernel around the three costs that dominated it and run it on four nodes. The contributions are:

- A dataflow kernel that processes one instruction slot per clock cycle for 64 shots (Section 4): a pipelined counter based noise stage, a Pauli frame in a one read, one write memory that completes every slot two cycles after reading it, and a detector fold stage that works on a per batch measurement record while the next batch is simulated.
- A compiler schedule that removes all frame hazards ahead of time: two slots on the same qubit are placed at least 8 slots apart, so the frame pipeline needs no hazard logic and still runs at the full rate (Section 4.6). On a distance 5 surface code the schedule adds only its 16 padding slots.

- A node model for independent sampling on several cards, in which every node, kernel instance and batch draws its own Threefry stream [14] and the host only launches work and collects results (Section 3).
- A verification chain from a software model through C simulation and RTL cosimulation to every card, closed by statistical comparison with Stim and by decoded logical error rates up to distance 7 (Sections 5 and 7.3).
- Measurements on one, two and four cards against Stim with one process and with all cores of the same host (Section 7.4).

Section 2 reviews related work, Section 3 describes the node model and Section 4 the kernel. Section 5 gives the verification method, Section 6 the experimental setup and Section 7 the results. Section 8 discusses limits and Section 9 concludes.

2 Related work

Stabilizer simulation.

The Gottesman Knill theorem makes Clifford circuits efficiently simulable through the Heisenberg picture of stabilizer groups [10]. Aaronson and Gottesman gave the tableau algorithm with destabilizers [11], and Anders and Briegel a graph state representation that they found faster than the tableau algorithm [15]. Stim [9] combines a tableau simulation of one reference shot with Pauli frame sampling of all other shots, vectorised over shots with SIMD instructions; it also derives the detector error model of a circuit, which decoders such as PyMatching consume [6, 7]. SymPhase accumulates possible Pauli faults as symbolic phases during a single pass over the circuit and draws samples by substituting values into the symbolic measurement outcomes; its authors report shorter sampling times than Stim on their benchmarks [16]. All of these run on CPUs.

Accelerated stabilizer simulation.

Osama, Thanos and Laarman reformulate tableau evolution and measurement for GPUs in the QuaSARQ simulator; they report speedups of up to 105 times on circuits with up to 180,000 qubits, consistently shorter run times than Stim, and an advantage in many shot sampling on large workloads [17]. quEStab extends the tableau to mixed Clifford and non Clifford circuits on several GPUs [18]. These GPU simulators are built around the tableau of wide circuits; the sampler of this paper instead streams the Pauli frames of many shots of a small QEC circuit through a fixed pipeline, and its output is the detector and observable bits that a decoding study consumes. We are not aware of an FPGA implementation of Pauli frame sampling other than our own first kernel (Section 4.2); searches of Crossref and arXiv for FPGA based stabilizer, Clifford or Pauli frame simulators found none.

FPGAs for quantum circuit simulation.

FPGA emulation of quantum circuits dates back two decades [19]. Statevector simulators on HBM equipped cards reach 30 qubits on one card [20], and our four card simulator distributes a statevector over the cards of one host and treats each card as a node [13]. Trefoil and Qu-Trefoil distribute much larger states over FPGAs attached

to SATA storage [21, 22]. Statevector simulation is limited by memory and by the exchange of amplitudes between devices; Pauli frame sampling is limited by neither, which is why it scales over nodes without communication.

FPGAs in quantum error correction.

Most FPGA work in QEC concerns decoding, where latency matters: lookup table decoders for small codes [23], distributed union find decoders [8, 24, 25], HeliosNet, which decodes lattice surgery operations on five FPGAs connected as a network [26], the Collision Clustering decoder on FPGA and ASIC hardware with megahertz decoding rates [27], and real time correction for the Shor and Steane codes [28]. These systems consume syndromes; the sampler of this paper produces them, for the offline studies that design codes and test decoders. The two are complementary, and a sampler that runs on the same class of device as a decoder can feed it test data at hardware rates.

High level synthesis.

The kernel is written in C++ and compiled with Vitis HLS [29]; Cong et al. review the methodology [30]. The design choices below (dataflow stages connected by streams, loops pipelined to one iteration per cycle, arrays partitioned into registers) are standard HLS constructs; the contribution is how the sampling algorithm is arranged so that these constructs reach full rate.

3 FPGA cards as nodes

The system is one host with four Alveo U55C cards [31] on PCIe, the same platform as our distributed statevector simulator [13]. As there, each card is a node: it runs its own copy of the kernel, owns its share of the work, and communicates only with the host. Figure 1 shows the arrangement. What differs is the work itself. In the statevector simulator a gate on a qubit that indexes the cards needs amplitudes of other nodes, and the exchange through host memory sets the scaling. In Pauli frame sampling the shots are independent, so a node needs nothing from any other node. The host's role shrinks to three tasks: compile the circuit once, load the same program onto every node, and collect the detector and observable bits that the nodes return.

Independent random streams.

Independence of the shots only holds if the random numbers of different nodes are independent. Every noise decision of the kernel is drawn from Threefry4x32-20 [14], a counter based generator: its output is a fixed function of a 128 bit counter and a 128 bit key, so any number of streams can be produced in parallel without shared state, and any single draw can be recomputed in software. Node c uses the key $(s, c+1, 0, 0)$ for a run seed s; kernel instance j of a node samples batches $j \cdot 2^{28}$ onwards; and batch g enters the counter. Two draws of the whole system therefore share a counter and key only if they belong to the same node, instance, batch, instruction and lane, that is, if they are the same draw. The streams are also reproducible: the software model

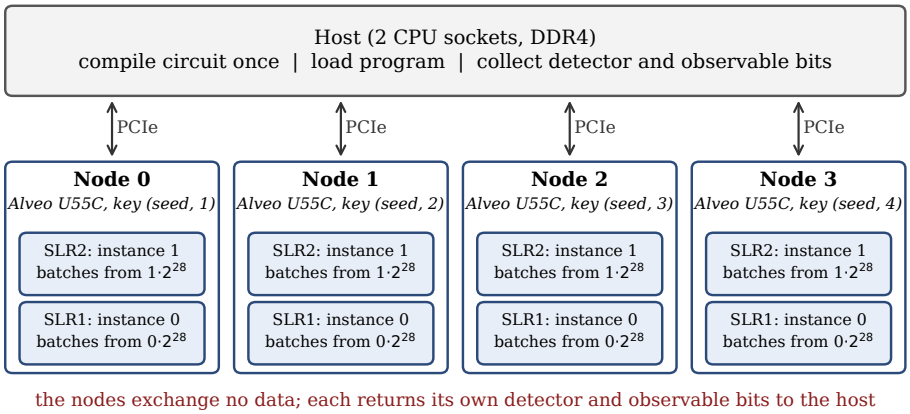


Fig. 1 Four Alveo U55C cards as nodes of one host. Every card holds 2 kernel instances, in super logic regions (SLRs) 1 and 2, each with its own HBM pseudo channels. The host compiles the circuit once, loads the same program into every instance and receives each instance's detector and observable bits; no data moves between cards.

of Section 5 recomputes the output of any node, instance and batch from the seed alone, which is how every card is checked bit for bit.

Host side.

One host process per node loads the bitstream, copies the program into every instance once and then keeps two launches per instance in flight: while one launch runs on the card, the output of the previous one is copied to host memory. A launch samples a fixed number of batches of 64 shots. For measurement, the processes of all nodes first load their cards and then wait at a common start barrier, so the timed interval of the system covers the same wall clock window for every node.

4 Kernel architecture

4.1 Pauli frame sampling

A Pauli frame assigns to every qubit q and shot a pair of bits (x_q, z_q), the X and Z components of the Pauli operator by which the noisy shot differs from the noiseless reference. Clifford gates map Pauli operators to Pauli operators, so the frame is updated by fixed bit rules: H swaps x_q and z_q, S sets $z_q \leftarrow z_q \oplus x_q$, and CX with control a and target b sets $x_b \leftarrow x_b \oplus x_a$ and $z_a \leftarrow z_a \oplus z_b$. A reset clears the frame of its qubit. A noise channel XORs a random Pauli into the frame. A measurement in the Z basis flips the shot's outcome relative to the reference exactly when $x_q = 1$, and the analogous rules hold in the X basis (z_q) and the Y basis ($x_q \oplus z_q$). A detector is the parity of a set of measurement outcomes whose noiseless parity is fixed, and the reference parity cancels when two shots are compared; the value of a detector in a shot is

therefore the XOR of the frame flips of its measurements. The kernel never needs the reference sample itself: it emits, for each detector and logical observable, whether the shot deviates from the noiseless value. This is the quantity Stim's detector sampler returns [9].

With L shots processed together, every frame component of a qubit is an L bit word and every rule above is one word wide bitwise operation. The kernel uses $L = 64$ lanes per instance.

4.2 The first kernel and its costs

The first kernel, built earlier in this project and not published, interprets a stream of instructions one at a time. Each step fetches a 41 byte instruction from HBM, applies it to a frame of 64 lanes, draws noise from a 64 lane Philox bank inside the same step, and on every measurement runs a loop over all 256 detector accumulators to apply the detector mask of that measurement. Its instruction loop was not pipelined (an attempt to pipeline it reached an initiation interval of 34 cycles and exceeded the resource budget), and each kernel launch sampled 64 shots. These costs, rather than any property of the algorithm, set its throughput: at a 250 MHz kernel clock it sampled the benchmark circuits of Section 6 at 865,117, 160,104 and 33,587 shots per second on one card, below one Stim process on an AMD EPYC 7742. Its output was nevertheless correct: on hardware its detector rates and its decoded logical error rates (up to distance 5) agreed with Stim, and single fault injection reproduced all 458 error mechanisms of the detector error models of three circuits. The kernel described next keeps the instruction semantics of the first kernel and its software model, and changes how they are executed.

4.3 Dataflow structure

Figure 2 shows the kernel. At the start of a launch the program and the fold list are copied from HBM into on chip memory once. Four stages then run concurrently, connected by FIFO streams, over all batches of the launch:

1. the noise stage reads one slot per cycle and computes the noise masks of its instruction for all lanes;
2. the frame stage applies one slot per cycle to the Pauli frame and emits the frame flip of every measurement;
3. the fold stage collects the measurement flips of a batch and computes every detector and observable as the XOR of its listed measurements;
4. the store stage writes the outputs of every batch to HBM in bursts.

Every loop of every stage is pipelined with an initiation interval of one cycle. The frame stage needs exactly one cycle per slot, and the other stages need fewer cycles per batch than it does, so a batch of 64 shots takes as many cycles as its program has slots.

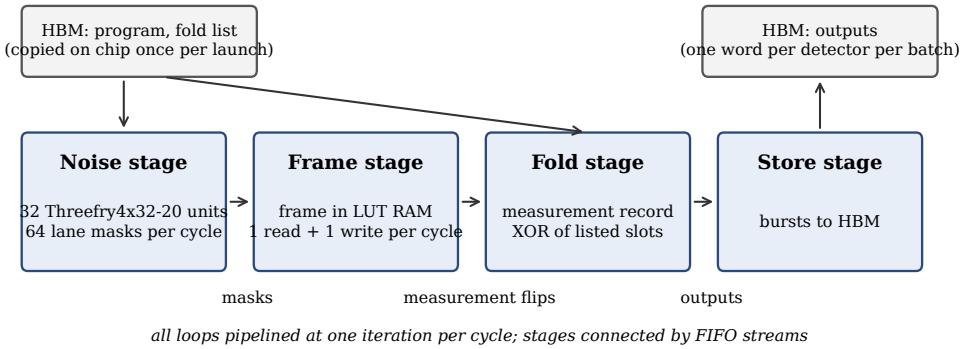


Fig. 2 Dataflow structure of one kernel instance. The noise stage evaluates 32 Threefry4x32-20 units per cycle; the frame stage keeps the frames in a one read, one write memory and completes every slot two cycles after its read; the fold stage keeps a per batch record of measurement flips. Stages are connected by FIFO streams and work on different instructions or batches at the same time.

4.4 Program and compiler

The compiler takes a Stim circuit and produces the same instruction list as for the first kernel: broadcast instructions are split into one instruction per qubit or qubit pair, REPEAT blocks are unrolled, and the measurement records that every DETECTOR and OBSERVABLE_INCLUDE refers to are resolved to absolute measurement indices. It then applies three transformations specific to this kernel. First, qubits are renumbered densely in order of first use: the generated surface code circuits use sparse qubit indices, and dense numbering lets a distance 7 surface code (97 qubits) fit the 128 entries of the frame. Second, every measurement receives a slot in the measurement record of a batch, and the detector and observable definitions become a fold list: for every output in order, the record slots of its measurements, with the last entry of each output marked. Third, the instructions are scheduled for the frame pipeline (Section 4.6). Every slot is encoded in 64 bits: opcode, the 7 bit index of the qubit it reads and writes, two flags marking the first and the second slot of a two qubit instruction, an 11 bit record slot and the 32 bit noise threshold $\lfloor p \cdot 2^{32} \rceil$. Table 1 lists the compiled programs.

4.5 Noise stage

The noise masks of an instruction depend only on the instruction, the lane and the batch, never on the frame. The noise stage therefore runs ahead of the frame stage and is decoupled from it by a stream. It uses a counter based generator from the family of Salmon et al. [14], whose members pass the TestU01 BigCrush battery: the output is a fixed function of a 128 bit counter and a key, so the generator has no state to carry and any draw can be recomputed in software. For an instruction whose (first) slot is i, lane l and batch g the stage evaluates Threefry4x32-20 with counter $(i, \lfloor l/2 \rfloor, g, 0)$ and the node's key. One evaluation yields four 32 bit words; even lanes use the first

two and odd lanes the last two, so 32 generator units serve 64 lanes per cycle. For lane l the first word w_0 decides whether the channel fires ($w_0 < t$ for the threshold t) and the second word w_1 selects which Pauli: $w_1 \bmod 3$ for single qubit depolarizing noise and $w_1 \bmod 15$ for two qubit depolarizing noise, as in the first kernel and its model. The modulo needs no divider: since $16 \equiv 1 \pmod{15}$, a 32 bit word and the sum of its eight hexadecimal digits are congruent modulo 15, the sum is at most 120, and one more digit sum and a conditional subtraction give the exact residue; $w_1 \bmod 3$ follows from $w_1 \bmod 15$. The twenty Threefry rounds are pipelined, giving the stage a depth of 25 cycles at one slot per cycle.

The first kernel used Philox4x32-10 from the same family, and so did the first versions of this kernel. Philox multiplies two 32 bit words in each of its ten rounds; 32 pipelined Philox units took 2,076 DSP blocks per instance, 69 % of those of one SLR. Threefry instead uses only additions, rotations and exclusive ors, which map to ordinary logic and carry chains: an instance needs 12 DSP blocks. Why this mattered is reported in Section 7.2. The software model computes the same Threefry function; it reproduces the known answer of the Random123 reference for counter and key zero and agrees with the ThreeFry generator of the randomgen package on random counters and keys.

4.6 Frame stage and hazard free scheduling

The frame stage must apply one instruction per cycle to a frame of up to 128 qubits, each with two words of 64 bits. We first held the frame in registers, so that a two qubit instruction could read both of its qubits and write both back in one cycle. That version reached one instruction per cycle in synthesis after the read, the gate logic and the write back were split into pipeline steps, but its implementation on the card failed: with two instances, the routed design failed Vivado's verification with conflicting nets after more than five hours of implementation. A congestion analysis of that design attributes the most congested short range windows to the frame stage, whose register crossbar (four 128 way selections and write paths into all frame registers) is local and dense, and the global and long range windows mostly to the noise stage, whose Philox multipliers fill the DSP columns of their region. The kernel therefore keeps the frame in a small memory with one read and one write per cycle, implemented in LUT RAM, and gives every slot exactly one frame read and one frame write. A single qubit instruction occupies one slot. A two qubit instruction occupies two consecutive slots: the first reads qubit a, the second reads qubit b, and their new values are written in the two following cycles. In each cycle the stage reads the qubit of slot k and completes slot $k-2$ from the operands read two and one cycles earlier; for the first slot of a pair it computes the new values of both qubits, writes a and keeps b, which the second slot writes one cycle later. A valid bit per qubit, cleared at the start of each batch, makes an unwritten qubit read as zero, so the memory never needs to be cleared. The synthesised frame loop runs at one slot per cycle with a depth of 5 cycles, and the instance needs 33 % of the LUTs of one SLR.

The price is a hazard: the result of slot k is written at the end of cycle $k+2$, so a slot that reads a qubit fewer than three slots after a slot that touched it sees a stale

value. The frame stage does not detect this. Instead the compiler guarantees that it never happens:

Proposition 1 *If every two slots that touch a common qubit are at least $D \geq 3$ apart, the frame stage computes the same frame, measurement flips and outputs as executing the instructions one at a time in slot order.*

Proof Slot k reads its qubit in cycle k, and the value of slot k' is written at the end of cycle $k' + 2$, so the read sees the result of every slot $k' \leq k - 3$ and of no later slot. Every earlier slot on the same qubit has $k' \leq k - D \leq k - 3$, so the value read is the one the sequential execution would read. For a pair occupying slots k and $k + 1$, the new values are computed in cycle $k + 2$ from the operands of both slots, which are correct by the same argument for each of the two qubits. Slots on disjoint qubits do not interact, and the measurement flip of a slot is computed from its correct operands. □

The compiler enforces $D = 8$, more than the three slots needed, as margin for the cycle in which the synthesised hardware performs the read and the write; RTL cosimulation of the repetition code, in which 36 of the 51 reuses of a qubit are at the minimum distance, confirms that the margin holds. The scheduler fills slots one by one: each slot, or pair of slots, takes the earliest instruction in compiler order whose qubits were last touched at least D slots before and whose predecessors on each of its qubits are already placed, or a NOP if there is none. The order of the instructions on every qubit is unchanged, and instructions on disjoint qubits commute, so the schedule computes the same frame as the compiler order. The program begins and ends with D NOPs, which write nothing, so that no write or measurement of one batch reaches past the clear of the valid bits at the start of the next. Two qubit instructions cost two slots, and NOPs cost slots only when a circuit has too little parallelism: Table 1 shows 16 NOPs, all of them padding, for the distance 5 surface code, and more for the small repetition code, whose five qubits cannot fill the slots between reuses.

The scheduler is checked by tests that every two slots on a qubit are at least D apart, that the two slots of a pair are consecutive, that the program begins and ends with D NOPs, and that the sequence of instructions on every qubit is identical to the compiler order. A negative control shows that the C simulation of the kernel models the delayed write: compiled with $D = 2$, a circuit whose every instruction depends on the one before produces output that differs from the software model.

4.7 Fold and store stages

The first kernel folded each measurement into the detectors immediately, which needs a read, modify and write of detector accumulators that two nearby measurements may share. Here the frame stage only sends the flip of every measurement, tagged with its record slot, to the fold stage, which writes it into a measurement record of the batch. When all measurements of the batch have arrived, the fold stage walks the fold list once, XORing the record entries of each output into an accumulator that is emitted at the last entry. The accumulator is the only state carried from one entry to the next, and the loop runs at one entry per cycle. The fold of batch g overlaps the frame

simulation of batch $g+1$, which needs a FIFO deep enough to hold the measurement flips produced meanwhile. The store stage writes the outputs of every batch to HBM in bursts: for each batch, one word of 64 bits per detector and per observable, bit l belonging to shot l of the batch.

4.8 Several instances per card

The Alveo U55C device has three super logic regions (SLRs); SLR0 also holds the platform's HBM controllers and DMA engine. A card holds 2 instances of 64 lanes, placed in SLR1 and SLR2 and connected to their own HBM pseudo channels through the SLR crossings. The instances of a card run independently, on disjoint batch ranges (Section 3).

5 Verification method

A sampler that is fast but subtly biased is worse than a slow one, because its errors propagate silently into threshold estimates. The kernel is therefore checked at every level of its implementation against a single software model, and the model is checked against Stim.

Software model.

The model interprets the compiled program in Python with one boolean frame per qubit and shot. Its gate, measurement, noise and fold semantics are those of the model of the first kernel, which was validated against Stim and against single fault injection from Stim's detector error model (Section 4.2); only the random stream of Section 4.5 is new. Given the program, the seed, the node key and the batch range, the model produces the exact output words the kernel must produce.

Bit exact levels.

The same comparison is applied at four levels, each catching a different class of error:

1. C simulation of the kernel source compiled with g++, for four circuits (repetition code, surface code memory in the Z and the X basis), two seeds and two batch offsets, one of them near the top of the 32 bit batch counter range. Noiseless circuits must give all zero outputs.
2. The negative control of Section 4.6, which shows that the C simulation would detect a schedule violation.
3. RTL cosimulation of the synthesised kernel with Vitis HLS, which runs the generated hardware description, including the pipelined frame stage and the concurrent dataflow stages: the repetition code, in which 36 of the 51 reuses of a qubit are at the minimum schedule distance, and 4 and 12 batches of the distance 3 surface code, which uses every gate and noise type of the benchmarks.
4. Every kernel instance of every card on hardware, for the three benchmark circuits, with the node key of that card and batch offsets in two launches.

Statistical levels.

Bit exactness against the model establishes that the hardware computes what the model computes. Agreement with Stim then establishes that the model samples the right distribution. Two tests are run on hardware samples. Tier 4 compares the firing rate of every detector and observable with Stim's sampler over 10,223,616 shots per side for the three benchmark circuits, using a two proportion z score per output and requiring $|z| < 5$ for all of them. Tier 5 decodes hardware samples and Stim samples of rotated surface code memory experiments with distance $d = 3, 5, 7$ (d rounds) and physical error rate $p = 0.001, 0.003, 0.01$ with the same PyMatching [6, 7] matcher, built from Stim's detector error model, and requires the logical error rates to agree with $|z| < 5$. Distance 7 could not be run on the first kernel, which lacked the qubit renumbering.

6 Experimental setup

Platform.

The host has two AMD EPYC 7742 processors with 256 hardware threads in total and 504 GiB of memory, and four Alveo U55C cards [31] with the platform `xilinx_u55c_gen3x16_xdma_base_3`, each in its own PCIe slot, driven by XRT 2.16.204 [32]. The kernel was synthesised with Vitis HLS 2023.2 [29] and linked with `v++` 2023.2 at a kernel clock of 300 MHz, with 2 instances per card.

Circuits.

The three benchmark circuits are those used to evaluate the first kernel (Section 4.2), generated by Stim: a repetition code memory experiment with distance 3 and 3 rounds (data and measurement noise 0.05), and rotated surface code memory experiments in the Z basis with distance 3 and 3 rounds and with distance 5 and 5 rounds, each with data depolarization before every round, measurement flips, depolarization after every Clifford gate and reset flips, all at probability 0.02. Table 1 lists the compiled programs.

CPU baseline.

Stim 1.13.0 runs on the same host. A Stim process compiles the detector sampler of the circuit (not timed) and then samples bit packed detector and observable data; the rate is the number of shots divided by the sampling time. Stim's sampler uses one thread per process, so the parallel baseline starts P processes behind a common barrier, each sampling its own share with its own seed, and divides the total number of shots by the time of the slowest process. We report $P = 1$ and P up to 256, the number of hardware threads of the host; every point is the median of 3 runs, taken while the cards were idle.

FPGA measurement.

For each circuit and for one, two and four cards, one host process per card loads its card and the program, waits at the common barrier, and then runs a fixed number

Table 1 Compiled programs. Instructions excludes the NOPs inserted by the schedule. Slots, the cycles per batch of the frame stage, counts one slot per single qubit instruction, two per two qubit instruction and one per NOP. Columns: qubits after renumbering, instructions, two qubit (2q) instructions, NOPs, slots, measurements, outputs (detectors plus observables) and fold entries.

Circuit	Qubits	Instr.	2q	NOPs	Slots	Meas.	Out.	Fold
Repetition code, $d = 3$	5	44	12	74	130	9	9	17
Surface code, $d = 3$	17	343	144	28	515	33	25	55
Surface code, $d = 5$	49	1,673	800	16	2,489	145	121	261
Surface code, $d = 7$	97	4,667	2,352	16	7,035	385	337	715

of launches of 4096 batches per instance, each launch's output copied back to host memory before it counts. The system rate is the total number of shots of all cards divided by the interval from the earliest start to the latest end of the timed loops. Each configuration is repeated 3 times and the median is reported. Before any timing, every instance of every card passes the bit exact check of Section 5. Card temperatures are recorded before and after every configuration.

7 Results

7.1 Compiled programs

Table 1 lists the compiled programs. The schedule of Section 4.6 adds no NOPs beyond the padding for the distance 5 and 7 surface codes, whose layers hold many independent instructions; their slot counts exceed the instruction counts by the second slots of the two qubit instructions. The distance 3 surface code needs a few NOPs and the repetition code, with five qubits, needs most of its slots as NOPs; both programs are short, so a batch still takes only a few hundred cycles.

7.2 Synthesis and implementation

All loops of the kernel were scheduled by Vitis HLS with an initiation interval of one cycle, and HLS estimated a maximum clock of 338 MHz for a target of 300 MHz. Placing and routing the kernel on the card was the difficult step. Table 2 lists every implementation run, each with two instances at 300 MHz. The 3 runs with the Philox noise stage all failed Vivado's verification of the routed design with conflicting nets, after about five hours each: first with the frame in registers, then with the frame in LUT RAM, and then also with a placement directive that spreads logic to relieve congestion. The congestion analysis of the first run (Section 4.6) explains the pattern: removing the register crossbar removed the short range hot spots, but the noise stage with 2,076 DSP blocks per instance still filled the DSP columns of its region, and in all three runs every one of the ten conflicting nets that Vivado lists lay in the instance placed in SLR0, which it shares with the platform's HBM and DMA logic. Replacing Philox by Threefry removed the DSP demand (12 DSP blocks per instance), and the instances were moved to SLR1 and SLR2. With both changes the design routed: the

Table 2 Implementation runs of the kernel on the Alveo U55C, two instances at a 300 MHz target. Frame: frame stage storage. Generator: Philox4x32-10 or Threefry4x32-20. SLRs: the regions of the two instances. Placement: Vivado directive ExtraTimingOpt (timing) or AltSpreadLogic_high (spread). DSP: DSP blocks per instance (HLS). Time: Vivado placement and routing. All runs used the same post route optimisation settings.

Frame	Generator	SLRs	Placement	DSP	Time	Outcome
registers	Philox	0, 1	timing	2,076	5 h 18 min	routing failed
LUT RAM	Philox	0, 1	timing	2,076	4 h 55 min	routing failed
LUT RAM	Philox	0, 1	spread	2,076	5 h 23 min	routing failed
LUT RAM	Threefry	1, 2	timing	12	2 h 04 min	bitstream
LUT RAM	Threefry	1, 2	spread	12	stopped	stopped

Table 3 Resources of the kernel. HLS: estimate for one instance, in percent of one SLR. Routed: one instance, and all 2 instances of a card in percent of the resources available to user logic, from the post route utilisation report.

Resource	HLS, one instance	% SLR	Routed, one instance	Routed, card (% budget)
LUT	145,414	33	80,941	161,975 (13.76)
FF	82,131	9	61,272	122,546 (5.06)
DSP	12	0	12	24 (0.27)

placer no longer reported the design as congested, and the run with the standard placement directive met timing at 300 MHz with a worst negative slack of 0.003 ns after 2 h 04 min of implementation, less than half the time of each failed run. The parallel run with the spreading directive was stopped once this result was available.

Table 3 compares the HLS estimate of one instance with the routed design. The HLS estimate of logic is conservative: a routed instance uses 80,941 LUTs against 145,414 estimated. The routed design runs at 300 MHz with a worst negative slack of 0.003 ns.

RTL cosimulation of the distance 3 surface code took 3,281 cycles for 4 batches and 7,412 cycles for 12, that is 516.4 cycles per additional batch for a program of 515 slots. The four stages keep pace with each other: the frame stage completes one slot per cycle, and a batch costs 1.4 cycles more than its slots.

7.3 Correctness on hardware

Every kernel instance of every card produced output bit identical to the software model for the three benchmark circuits (12 of 12 card and circuit checks). Table 4 gives the Tier 4 comparison with Stim and Table 5 the Tier 5 logical error rates. All outputs of all circuits agree with Stim within the 5σ bound (largest $|z|$ 3.12), and the decoded logical error rates of hardware and Stim samples agree at every distance and error rate (largest $|z|$ 1.25). The rates show the expected behaviour of the surface code: below threshold the logical error rate falls with distance, at $p = 0.01$ it rises.

Table 4 Tier 4: detector and observable firing rates of hardware samples against Stim. The largest two proportion $|z|$ over all outputs of a circuit is reported.

Circuit	FPGA shots	Stim shots	max $\|z\|$
Repetition code, $d = 3$	10,223,616	10,000,000	1.43
Surface code, $d = 3$	10,223,616	10,000,000	3.12
Surface code, $d = 5$	10,223,616	10,000,000	2.74

Table 5 Tier 5: logical error rates (LER) of rotated surface code memory experiments, hardware samples against Stim samples, both decoded by the same PyMatching matcher.

d	p	Shots	FPGA LER	CPU Stim LER	z
3	0.001	1,048,576	7.753e-04	7.734e-04	+0.05
3	0.003	1,048,576	6.518e-03	6.616e-03	−0.87
3	0.010	1,048,576	5.939e-02	5.908e-02	+0.97
5	0.001	1,048,576	1.402e-04	1.373e-04	+0.18
5	0.003	1,048,576	3.340e-03	3.286e-03	+0.67
5	0.010	1,048,576	8.339e-02	8.377e-02	−0.99
7	0.001	1,048,576	1.621e-05	2.289e-05	−1.09
7	0.003	1,048,576	1.466e-03	1.533e-03	−1.25
7	0.010	1,048,576	1.034e-01	1.032e-01	+0.39

Table 6 Sampling rates in shots per second (median of 3 runs) on one, two and four cards. S_4: four card rate over one card rate. R_1, R_{all}: four card rate over Stim on the same host with one process and with 256 processes (Table 7). All rates count only shots whose outputs have reached host memory.

Circuit	1 card	2 cards	4 cards	S_4	R_1	R_{all}
Repetition code, $d = 3$	289,230,495	577,517,899	1,155,366,309	3.99×	193×	8.24×
Surface code, $d = 3$	73,912,169	147,798,429	295,658,952	4.00×	104×	2.39×
Surface code, $d = 5$	15,338,202	30,687,981	61,359,029	4.00×	84.8×	1.07×

7.4 Throughput on one, two and four nodes

Table 6 and Figure 3 give the measured sampling rates. One card with 2 instances samples the distance 5 surface code at 15.3 million shots per second, and four cards at 61.4 million. Going from one to four nodes multiplies the rate by 3.99 to 4.00 over the three circuits, that is, the nodes scale linearly: they share nothing but the host, and at these output volumes the host side (launching, copying results to host memory, synchronising the processes) does not limit the rate. The per card rates within a run differ by less than one percent. The FPGA temperatures read before and after every configuration were at most 72 °C.

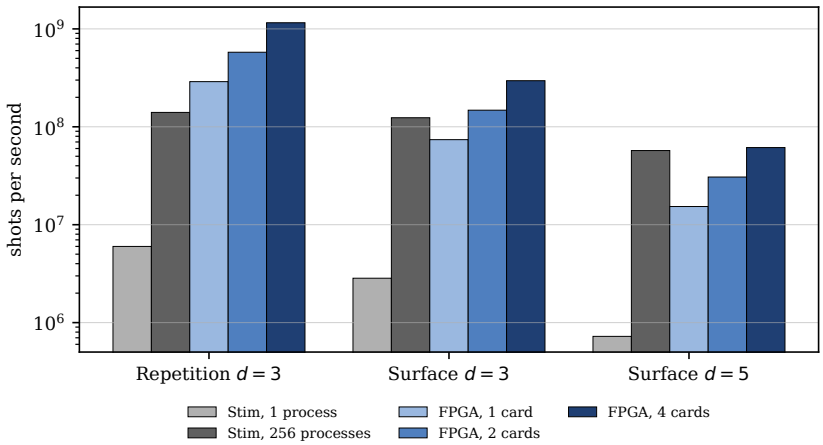


Fig. 3 Sampling rate of the three benchmark circuits on one, two and four cards, and of Stim with one process and with all hardware threads of the host. Logarithmic scale.

Table 7 Stim 1.13.0 on the host: shots per second with P concurrent processes.

Circuit	1 proc.	16 proc.	64 proc.	128 proc.	256 proc.
Repetition code, $d = 3$	5,999,732	60,814,996	84,704,275	100,987,950	140,258,256
Surface code, $d = 3$	2,841,093	35,270,693	65,480,163	78,332,218	123,750,688
Surface code, $d = 5$	723,625	10,181,623	26,614,558	35,169,041	57,200,963

Against one Stim process, the four node system is faster by the factors in Table 6 (84.8 times for the distance 5 surface code). The more demanding comparison is with Stim on all 256 hardware threads of the same host: there the four node system reaches 1.07 times the CPU rate for the distance 5 surface code, 2.39 times for the distance 3 surface code and 8.24 times for the repetition code. Stim's rate grows less than linearly with the number of processes (Table 7); we did not investigate the cause on this host.

7.5 Comparison with the first kernel and with the cycle model

On one card the first kernel sampled the three circuits at 865,117, 160,104 and 33,587 shots per second (Section 4.2), measured at a 250 MHz kernel clock on the project's build host, whose CPU is the same model. The new kernel on one card is faster by factors of 334, 462 and 457. The cycle count of the kernel predicts the rate of a card from the program alone: an instance delivers 64 shots per batch, and a batch takes as many cycles as the largest of its slot count, its measurement and fold work, and its output count, so a card should sample at 295 million, 74.6 million and 15.4 million shots per second. The measured one card rates are 98 %, 99 % and 99 % of these predictions; the remainder is launch overhead and the copy of results to host memory, which weigh most for the short repetition code program.

8 Discussion

What changed.

The first kernel and the new one execute the same instructions with the same frame semantics and the same noise decision rules; they differ in the counter based generator that supplies the random words, and in where the work happens: noise generation moved off the critical loop into its own pipelined stage, frame hazards moved from hardware into the compiler's schedule, detector folding moved into a stage that overlaps the next batch, and the cost of a launch is shared by thousands of batches instead of one. Each of these is a standard technique; together they turn the interpreter of the first kernel into a pipeline whose rate the compiled program alone determines.

Nodes without communication.

Our distributed statevector simulator on the same four cards spends much of its time exchanging amplitudes through host memory, and more cards add capacity there rather than speed [13]. Sampling is the opposite case: with independent random streams per node the nodes do not interact, and the four card rate is four times the one card rate (Table 6). The node model carries over unchanged; what decides scaling is whether the workload needs data from other nodes. For workloads of this kind, more cards in a host, or hosts with cards, add rate in proportion until the host can no longer collect the output.

Routability as the limit.

The synthesis estimates were never the obstacle: every version of the kernel that was implemented reached one slot per cycle and an estimated clock above the 300 MHz target. What decided between a working and a failing design was routing, and the cause was density rather than size: the failed designs used 46 % of the DSP blocks available to user logic but packed them into the DSP columns of the regions they occupied, and the conflicting nets of every failed run lay in the instance that shared SLR0 with the platform logic. A generator built from adders and rotations, which spreads over ordinary logic, and the move away from SLR0 turned five hour runs that failed into a two hour run that met timing. For HLS designs that replicate arithmetic units in parallel, we found the placement of those units on the device to matter as much as their number.

Limits of the kernel.

The frame memory holds 128 qubits after renumbering, enough for the rotated surface code up to distance 7 (97 qubits) but not distance 9. The program memory holds 8,192 slots, the measurement record 2,048 measurements and the output stage 512 detectors and observables. Larger codes need wider limits; the frame memory grows linearly with the qubit count, and the program and record memories are on chip block RAM. The instruction set covers the gates, measurements, resets and noise channels of the generated memory experiments (H, S, CX, CZ, SWAP, measurement and reset in three bases, measure and reset, single qubit Pauli errors, single and two qubit depolarizing noise); other Clifford gates, correlated errors and Pauli product measurements would

need further opcodes. Like the first kernel, it uses the frame semantics that follow from a Clifford circuit; it does not handle non Clifford gates.

Limits of the comparison.

The CPU baseline is Stim's own detector sampler with one process per hardware thread, which is how Stim is used for large sampling campaigns; a multi threaded sampler built on Stim might do better on the same CPU. The comparison concerns sampling only: compiling the circuit, deriving the detector error model and decoding are done on the host in both cases and are not timed. Energy was not compared, because the host provides no power measurement of the CPU that would match the board power readings of the cards. The repetition code has only five qubits, so most slots of its program are NOPs that keep reuses of a qubit apart (Table 1); its rate reflects the schedule more than the circuit, and the larger circuits are the more representative ones.

Embedded use.

The kernel needs no host interaction while a launch runs and its output is a stream of detector words. The same pipeline could therefore feed a decoder on the same device or on a device next to it, replacing recorded test data with live samples at hardware rate, which would make the sampler a test source for the FPGA decoders of Section 2. We have not built that combination.

9 Conclusions

We presented a Pauli frame sampler for stabilizer circuits that processes one instruction slot per clock cycle for 64 shots per kernel instance, and ran it on four Alveo U55C cards treated as independent nodes of one host. A pipelined counter based noise stage, a one read, one write frame pipeline whose hazards the compiler schedules away, and a detector fold that overlaps the next batch remove the costs that made the first kernel slower than one CPU process. The kernel is bit exact against its software model in C simulation, in RTL cosimulation and on every card, and its samples agree with Stim in detector rates and in decoded logical error rates up to distance 7. On four nodes it samples the distance 5 surface code at 61.4 million shots per second, 84.8 times one Stim process and 1.07 times Stim on all 256 hardware threads of the host.

The routed design with two instances occupies 13.76 % of the LUTs available to user logic and almost no DSP blocks, which leaves room for more instances per card. The next steps are such wider configurations, wider limits for larger codes, and a direct connection of the sampler to an FPGA decoder so that decoders can be tested with live samples.

Acknowledgments. This work was funded by the Ministry of Electronics and Information Technology (MeitY), Government of India. The research was conducted at the C-DAC Noida Centre under the supervision of Mr. Abhishek Tiwari, Scientist F, Embedded Systems Division, C-DAC Noida. We thank the developers of Stim and PyMatching, on which the validation of this work depends.

Declarations

Funding. This work was funded by the Ministry of Electronics and Information Technology (MeitY), Government of India.

Conflict of interest. The author declares no competing interests.

Ethics approval and consent to participate. Not applicable.

Consent for publication. Not applicable.

Data availability. The raw measurement records that support the findings of this study (one JSON file per run with timestamps, code revision, bitstream hash and card identifiers), the benchmark circuits and the scripts that generate every table and figure from them are available from the corresponding author on reasonable request.

Author contribution. N. Ali designed and implemented the HLS kernel, the compiler, the host runtime and the validation suite, carried out the measurements and wrote the manuscript.

References

[1] Kitaev, A.Y.: Fault-tolerant quantum computation by anyons. Annals of Physics **303**(1), 2–30 (2003) https://doi.org/10.1016/s0003-4916(02)00018-0

[2] Dennis, E., Kitaev, A., Landahl, A., Preskill, J.: Topological quantum memory. Journal of Mathematical Physics **43**(9), 4452–4505 (2002) https://doi.org/10.1063/1.1499754

[3] Fowler, A.G., Mariantoni, M., Martinis, J.M., Cleland, A.N.: Surface codes: Towards practical large-scale quantum computation. Physical Review A **86**(3), 032324 (2012) https://doi.org/10.1103/physreva.86.032324

[4] Google Quantum AI: Suppressing quantum errors by scaling a surface code logical qubit. Nature **614**(7949), 676–681 (2023) https://doi.org/10.1038/s41586-022-05434-1

[5] Google Quantum AI and Collaborators: Quantum error correction below the surface code threshold. Nature **638**(8052), 920–926 (2025) https://doi.org/10.1038/s41586-024-08449-y

[6] Higgott, O.: Pymatching: A python package for decoding quantum codes with minimum-weight perfect matching. ACM Transactions on Quantum Computing **3**(3), 1–16 (2022) https://doi.org/10.1145/3505637

[7] Higgott, O., Gidney, C.: Sparse blossom: correcting a million errors per core second with minimum-weight matching. Quantum **9**, 1600 (2025) https://doi.org/10.22331/q-2025-01-20-1600

[8] Delfosse, N., Nickerson, N.H.: Almost-linear time decoding algorithm for topological codes. Quantum **5**, 595 (2021) https://doi.org/10.22331/q-2021-12-02-595

[9] Gidney, C.: Stim: a fast stabilizer circuit simulator. Quantum **5**, 497 (2021) https://doi.org/10.22331/q-2021-07-06-497

[10] Gottesman, D.: The heisenberg representation of quantum computers. arXiv preprint arXiv:quant-ph/9807006 (1998) arXiv:quant-ph/9807006

[11] Aaronson, S., Gottesman, D.: Improved simulation of stabilizer circuits. Physical Review A **70**(5), 052328 (2004) https://doi.org/10.1103/physreva.70.052328

[12] Knill, E.: Quantum computing with realistically noisy devices. Nature **434**(7029), 39–44 (2005) https://doi.org/10.1038/nature03350

[13] Ali, N.: Multi-FPGA Quantum Circuit Simulation: A Distributed Statevector Architecture on Four Xilinx Alveo U55C Accelerators. Research Square preprint. Under major revision, The Journal of Supercomputing (2026). https://doi.org/10.21203/rs.3.rs-10128153/v1

[14] Salmon, J.K., Moraes, M.A., Dror, R.O., Shaw, D.E.: Parallel random numbers. In: Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–12 (2011). https://doi.org/10.1145/2063384.2063405

[15] Anders, S., Briegel, H.J.: Fast simulation of stabilizer circuits using a graph-state representation. Physical Review A **73**(2), 022334 (2006) https://doi.org/10.1103/physreva.73.022334

[16] Fang, W., Ying, M.: Symphase: Phase symbolization for fast simulation of stabilizer circuits. In: Proceedings of the 61st ACM/IEEE Design Automation Conference, pp. 1–6 (2024). https://doi.org/10.1145/3649329.3655902

[17] Osama, M., Thanos, D., Laarman, A.: Gpu-accelerated quantum simulation of stabilizer circuits. Quantum **10**, 2225 (2026) https://doi.org/10.22331/q-2026-10-01-2225

[18] Shin, H., Lee, S., Kwak, M., Kim, Y.: questab: Towards scalable quantum circuit simulation on multi-gpu using an extended stabilizer formalism. In: Proceedings of the 40th ACM International Conference on Supercomputing, pp. 1296–1309 (2026). https://doi.org/10.1145/3797905.3816723

[19] Khalid, A.U., Zilic, Z., Radecka, K.: Fpga emulation of quantum circuits. In: IEEE International Conference on Computer Design: VLSI in Computers and Processors, 2004. ICCD 2004. Proceedings., pp. 310–315 (2004). https://doi.org/10.1109/iccd.2004.1347938

[20] Tiwari, A., Ali, N., Khan, N., Mazhar, I., Islur, K., Gupta, H., Kumar, S., Tyagi, P.: FPGA Based Quantum Circuit Simulator: Processing 30 Qubit Systems. Research Square preprint. Under major revision, The Journal of Supercomputing

(2026). https://doi.org/10.21203/rs.3.rs-8581344/v1

[21] Niwase, R., Harasawa, H., Yamaguchi, Y., Wei, K., Amano, H., Miyoshi, T.: Enormous-scale quantum state vector calculation with fpga-accelerated sata storages. In: 2023 International Conference on Field Programmable Technology (ICFPT), pp. 288–289 (2023). https://doi.org/10.1109/icfpt59805.2023.00049

[22] Wei, K., Amano, H., Niwase, R., Yamaguchi, Y., Miyoshi, T.: Qu-trefoil: Large-scale quantum circuit simulator working on fpga with sata storages. IEEE Transactions on Computers **74**(4), 1306–1321 (2025) https://doi.org/10.1109/tc.2024.3521546

[23] Das, P., Locharla, A., Jones, C.: Lilliput: a lightweight low-latency lookup-table decoder for near-term quantum error correction. In: Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 541–553 (2022). https://doi.org/10.1145/3503222.3507707

[24] Liyanage, N., Wu, Y., Deters, A., Zhong, L.: Scalable quantum error correction for surface codes using fpga. In: 2023 IEEE International Conference on Quantum Computing and Engineering (QCE), pp. 916–927 (2023). https://doi.org/10.1109/qce57702.2023.00106

[25] Liyanage, N., Wu, Y., Tagare, S., Zhong, L.: Fpga-based distributed union-find decoder for surface codes. IEEE Transactions on Quantum Engineering **5**, 1–18 (2024) https://doi.org/10.1109/tqe.2024.3467271

[26] Liyanage, N., Wu, Y., Houghton, E., Zhong, L.: Multi-fpga system for quantum error correction with lattice surgery. In: 2024 IEEE International Conference on Quantum Computing and Engineering (QCE), pp. 622–623 (2024). https://doi.org/10.1109/qce60285.2024.10435

[27] Barber, B., Barnes, K.M., Bialas, T., Buğdaycı, O., Campbell, E.T., Gillespie, N.I., Johar, K., Rajan, R., Richardson, A.W., Skoric, L., Topal, C., Turner, M.L., Ziad, A.B.: A real-time, scalable, fast and resource-efficient decoder for a quantum computer. Nature Electronics **8**(1), 84–91 (2025) https://doi.org/10.1038/s41928-024-01319-5

[28] Ali, N.: FPGA-Based Real-Time Quantum Error Correction for Shor and Steane Codes. Research Square preprint. Under major revision (2026). https://doi.org/10.21203/rs.3.rs-10115324/v1

[29] Advanced Micro Devices: Vitis High-Level Synthesis User Guide (UG1399), version 2023.2. https://docs.amd.com/r/en-US/ug1399-vitis-hls (2023)

[30] Cong, J., Liu, B., Neuendorffer, S., Noguera, J., Vissers, K., Zhang, Z.: High-level synthesis for fpgas: From prototyping to deployment. IEEE Transactions

on Computer-Aided Design of Integrated Circuits and Systems **30**(4), 473–491 (2011) https://doi.org/10.1109/tcad.2011.2110592

[31] Advanced Micro Devices: Alveo U55C Data Center Accelerator Card Data Sheet (DS978). https://docs.amd.com/r/en-US/ds978-u55c (2023)

[32] Advanced Micro Devices: Xilinx Runtime (XRT) Documentation. https://xilinx.github.io/XRT/ (2023)